# APK Intelligence CLI: Master Specification & AI Build Prompt

## Instructions for the AI Code Generator

You are an expert Python security tools developer. Your task is to build the "APK Intelligence CLI" based exactly on the specifications in this document. Do not deviate from the tech stack. Do not add dynamic analysis (no emulators, no hooking). Write modular, heavily commented Python code suitable for a Linux terminal.

## 1. System Overview

The APK Intelligence CLI is a pure Python static analysis tool for Linux. Its objective is to orchestrate the jadx command-line decompiler, use Python's built-in multi-threading to rapidly scan the decompiled files, match findings against a dynamically updatable local SQLite database, and generate a deterministic risk score.

## 2. Technology Stack

- **Language:** Python 3 (Strictly standard library where possible to avoid dependency bloat).
- **Decompiler:** jadx (Command-line tool, executed via Python subprocess).
- **Database:** sqlite3 (Built-in Python module for offline threat intelligence).
- **Network Requests:** urllib.request (Built-in Python module to fetch updated threat feeds without requiring pip install requests).
- **Concurrency:** concurrent.futures.ThreadPoolExecutor (Required to scan thousands of decompiled text files simultaneously).
- **Terminal UI:** rich (For rendering color-coded tables and progress indicators).
- **Data Handling:** re (Regex for finding IPs/URLs), xml.etree.ElementTree (Manifest parsing), json, csv (Exports).

## 3. Step-by-Step Execution Flow

### Pre-Requisite Utility: Threat Feed Updater (core/updater.py)

- This is a standalone script run periodically by the user to prevent database rot.
- **Network Fetch:** Use urllib.request to download raw text files from public Open-Source Intelligence (OSINT) feeds (e.g., URLhaus, Abuse.ch).
- **Database Update:** Connect to threat_intel.db. Delete outdated records in the malicious_network table. Insert the newly downloaded IPs and domains.

### Step 1: Input and Validation (apkintel.py)

- Use argparse to accept the APK file path and optional flags (--export json, --export csv,

--update-db).
- Verify the target file exists and is a valid ZIP/APK archive.
- Verify the jadx binary is available in the system PATH.
- Verify the threat_intel.db (SQLite) exists in the working directory.

## Step 2: Full Decompilation (core/decompiler.py)

- Create a temporary directory (e.g., /tmp/apkintel_<random>).
- Use subprocess.run to execute: jadx -d <temp_dir> <target.apk>.
- Wait for the process to complete successfully.

## Step 3: High-Speed Data Extraction (core/extractor.py)

- **Manifest Parser:** Read /tmp/<temp_dir>/resources/AndroidManifest.xml. Extract package_name, version_name, and all <uses-permission> tags.
- **Source Code Scanner:** Use os.walk to find all .java files.
- **Concurrency:** Use ThreadPoolExecutor to open and read multiple .java files concurrently.
- **Regex Targets:**
  - IPv4 Addresses.
  - URLs (http/https).
  - Known Tracker Signatures (e.g., com.google.firebase, com.facebook.appevents, com.appsflyer).
  - Suspicious APIs (e.g., dalvik.system.DexClassLoader for dynamic loading, java.lang.reflect for reflection).

## Step 4: Threat Matching (core/analyzer.py)

- Connect to threat_intel.db.
- The database contains a table malicious_network with columns indicator (IP/Domain) and type (malware, phishing, tracker).
- Query the database to check if any extracted IP or URL matches a known threat.

## Step 5: Deterministic Scoring Engine (core/scorer.py)

Start with a Risk Score of 0. Max is 100.

- **Critical Permissions:** +15 points each (e.g., READ_SMS, READ_CONTACTS, RECORD_AUDIO, ACCESS_FINE_LOCATION).
- **High-Risk APIs:** +10 points each (e.g., Reflection, Dynamic Loading).
- **Known Trackers:** +5 points each.
- **Malicious DB Match:** +30 points.
- *Verdict logic:* 0-30 = Low Risk, 31-60 = Medium Risk, 61-100 = High Risk.

## Step 6: Reporting and Cleanup (core/reporter.py)

- Use rich.table.Table to output the Metadata, Verdict, extracted IOCs (IPs/Domains), and suspicious behaviors in the terminal.
- If --export is used, dump the exact same data dictionary to the requested file format.

- Use shutil.rmtree to forcefully delete the temporary JADX directory upon completion or fatal error.

# 4. Expected Output Format (JSON Structure for reference)


```
{
  "metadata": { "package": "com.example.app", "version": "1.0" },
  "verdict": { "score": 75, "severity": "High" },
  "permissions": ["android.permission.READ_SMS"],
  "network": { "ips": ["192.168.1.1"], "domains": ["evil.com"] },
  "trackers": ["Firebase"],
  "capabilities": ["Reflection", "Dynamic Loading"]
}
```